

Project Report: Parallelization with OpenMP

Group 20

Rodrigo Perestrelo (106074)

Eduardo Silva (106929)

Gonçalo Jesus (96864)

1. Introduction and Analysis of the Serial Version

This project focuses on optimizing a clustering algorithm that associates different *Documents* to *Cabinets*, minimizing the differences in their score arrays (*Subjects*). The main goal was to improve performance through parallelization using OpenMP (multithreading and vectorization), removing bottlenecks tied to nested loops and repetitive Euclidean distance calculations.

1.1 Serial Version Bottleneck Identification

By analyzing the original code, the algorithm is essentially a variation of *K-Means clustering*. The main function `associate_cabin` iteratively allocates documents until stability (`changed == false`) is reached. The main "bottlenecks" (performance limiters) of the code are located in the operations performed during this iterative convergence:

1. **Calculating Cabinet Averages:** Two nested loops update the sum and division of the scores, iterating dynamically over all documents.
2. **Calculating Distances:** The triple nested loop (*Documents* $\rightarrow$ *Cabinets* $\rightarrow$ *Subjects*) where the Euclidean distance is calculated. For large test sets, this part performs intensive floating-point difference calculations, consuming the vast majority of the sequential execution time.

2. The Parallelization Process

2.1 The Approach Used for Parallelization

The chosen parallelization strategy was **Data Parallelism**. Since calculating the classes' means and evaluating new associations for the documents can be modeled as workflows that iterate over massive independent data structures, injecting the shared-memory paradigm via OpenMP was optimal.

2.2 What Decomposition Was Used

We used a **Data/Domain Decomposition**. During the calculation of the cabinets' "centers" and the distance calculations, the iteration space of the dimensional iterators (*Documents* or *Cabinets*) is subdivided into chunks and handled by the multiple active threads. Functional decomposition was not utilized.

2.3 What Were the Synchronization Concerns and Why

Introducing multithreading into the program immediately causes *Race Conditions* if global arrays (`cabins_size`, `cabins_values`) are modified simultaneously. We addressed these concerns through:

- **Thread-Local Buffers (Delta Size):** In our SIMD version, we circumvented massive lock congestion on `cabins_size` by giving each thread a private array segment (`size_deltas`). Threads tally their cabinet allocations locally and run a parallel reduction at the barrier, drastically eliminating shared memory contention.
- **Reduction Clauses:** We utilized `#pragma omp parallel for reduction(+:...)` for `cabins_values` arrays, allowing local variable accumulation that OpenMP safely collapses.
- `reduction(||:changed):` The `changed` stability flag was synced securely via a logical OR reduction so the system only transitions to a new iteration when all threads agree stability hasn't been met.

2.4 How Was Load Balancing Addressed

Since the computational load within each `for` iteration is strictly constant, OpenMP's default `schedule(static)` works perfectly, dealing linear blocks of iteration to threads with functionally zero overhead.

3. Application of SIMD (Single Instruction, Multiple Data)

Definition: SIMD is a hardware-level data parallelism technique, whereby a single physical processing unit executes the exact same operation simultaneously over multiple data elements using long CPU vector registers.

Implementation: In the `docs-omp.cpp` file, we applied `#pragma omp simd`, reorganized variables, hoisted pointer offsets (`doc_ptr`, `cab_ptr`), and enabled aggressive compiler optimizations (`-Ofast -fopenmp-simd -ffast-math`).

- **Relevant Code Modifications:** We used parallel array zeroing to avoid overheads. During average computations, we extracted division via an inverse multiplication loop: `double inv = 1.0 / cabins_size[cabin];` to retain uninterrupted vectorization. Distance calculations rely directly on `#pragma omp simd reduction(+:current_dist)` to fill SIMD lanes effectively.

4. Test Environment

- **Base Systems:** Lab 6 PC - P8.
- **Methodology:** We used our custom bash scripts to execute the binaries across varied target dimensions (`ex1000-50000-200.in`, `ex1000-50000-5-.2.in`, `ex40...`, etc.) using `#Threads` equal to 1, 2, 4, and 6.

5. Performance Results

*Note: The ideal Speedup is equal to the number of threads (T). All benchmarks were executed on **lab6p8**. Bolded values indicate the best performance for each input.*

tableExecution Times Comparison (lab6p8)			
Test Input	Serial	OMP Base (1/2/4/6t)	OMP + SIMD (1/2/4/6t)
ex1000-50000-200	203.63s	197.4s / 100.4s / 50.7s / 34.6s	42.98s / 22.23s / 11.61s / 8.18s
ex40-100000-20	34.42s	32.29s / 16.49s / 8.45s / 5.91s	14.30s / 7.19s / 3.74s / 2.77s
ex1000-50000-5	123.77s	146.3s / 74.44s / 37.75s / 25.83s	451.5s / 229.5s / 116.1s / 78.71s

5.1 Speedup Calculations

Test Case: ex1000-50000-200 (Serial: 203.63s)

OMP Base: 1.03x (1t) | 2.03x (2t) | 4.02x (4t) | 5.89x (6t)

OMP+SIMD: 4.74x (1t) | 9.16x (2t) | 17.54x (4t) | **24.89x (6t)**

Note: Multiplicative effect between SIMD and multithreading. At 6 threads, the SIMD version is $\sim 4.2x$ faster than the OMP Base version, achieving a total speedup of 24.89x.

Test Case: ex40-100000-20 (Serial: 34.42s)

OMP Base: 1.07x (1t) | 2.09x (2t) | 4.08x (4t) | 5.82x (6t)

OMP+SIMD: 2.41x (1t) | 4.79x (2t) | 9.19x (4t) | **12.45x (6t)**

Note: Strong vectorization gains even with 20 subjects. Performance scales almost linearly with the number of threads for both versions.

Test Case: ex1000-50000-5 (Serial: 123.77s)

OMP Base: 0.85x (1t) | 1.66x (2t) | 3.28x (4t) | **4.79x (6t)**

OMP+SIMD: 0.27x (1t) | 0.54x (2t) | 1.07x (4t) | 1.57x (6t)

Note: Severe SIMD regression. The vectorized version at 1t is $\sim 3.7x$ slower than the serial version. SIMD only breaks even with the serial baseline at 4 threads, but remains far inferior to OMP Base.

5.2 Analysis of the Results

1. **Superlinear Speedups with SIMD+Threading:** The most striking results appear in ex1000-50000-200 and ex40-100000-20 at 6 threads with SIMD, achieving 24.89x and 12.45x respectively — far exceeding the ideal linear limit of 6x. This superlinear behaviour is not a measurement artifact: vectorization introduces a multiplicative acceleration factor on top of thread parallelism. With 200 subjects per distance calculation, SIMD lanes are fully utilised, and the combined effect of 6 threads each executing wide vector operations compounds substantially beyond what either technique achieves alone.
2. **Thread Scaling and Efficiency:** For the SIMD versions, 6 threads represents a high performance peak, with ex1000-50000-200 scaling from 4.74x at 1t to 24.89x at 6t. Unlike previous environments, the workload remains compute-bound enough to avoid sharp regressions up to 6t. The OMP Base versions also scale near-linearly (5.89x at 6t), as the workload per thread remains substantial enough to keep all

cores productive. However, the performance gap between OMP Base and SIMD remains massive (over 4x difference), confirming that SIMD is the primary driver of efficiency for large subject counts.

3. **SIMD Regression for Small Subject Counts:** When the subject dimension is 5 (ex1000-50000-5), SIMD performs significantly worse than serial, showing only 0.27x speedup at 1 thread. Even at 6 threads, it barely reaches 1.57x, failing to outperform the serial baseline significantly. With only 5 elements per vector operation, the SIMD lanes are nearly empty, and the overhead of the vectorized loop structure — setup, alignment checks, and reduction bookkeeping — far outweighs any benefit. Effective vectorization requires wide, contiguous data streams; a subject count of 5 clearly does not satisfy this condition.
4. **Results vs. Expectations:** The OMP Base speedups were broadly in line with expectations: near-linear scaling up to 6 threads, consistent with Amdahl’s Law applied to a mostly-parallel workload. What exceeded expectations was the magnitude of the SIMD gains for large subject counts: a speedup of 24.89x at 6 threads is significantly beyond the theoretical linear limit, confirming that vectorization and threading compound multiplicatively rather than additively when the data is wide enough to fill hardware SIMD lanes. Conversely, the severity of the SIMD regression for 5 subjects was a negative surprise; a near 73% slowdown relative to the serial baseline at 1 thread (0.27x) highlights how critically SIMD performance depends on the inner loop dimension.